Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Strategy kalıbı (ekrandan, 00:24:30, 00:30:00)
  - Amaç: aynı iş için kullanılabilecek farklı algoritmaları, işi isteyenden soyutlamak. Böylece kullanılan algoritma çalışma anında değiştirilebilir; iş ile algoritma gevşek bağlanır (loosely coupled).
  - Dörtlü Çete'nin örneği: kelime işlemcide satır bölmeleme (line breaking). Kaydırma yok, pencereye göre kaydırma, cetvele göre kaydırma, boşluktan ya da elle konan heceleme işaretinden bölme gibi farklı yöntemler vardır. Hoca bunu WordPad'deki "Sözcük kaydır" seçenekleriyle gösterdi (ekrandan, 00:20:00).
  - Çözüm: soyut bir satır bölmeleme sınıfı (ya da arayüzü) ve onun soyut `bölmele` metodu; her bölmeleme yöntemi bir somut alt sınıf. Uygulama işi "bölmeleyici" üye alanı üzerinden yaptırır; bölmeleme biçimini değiştirmek için bu üyeye başka bir somut nesne verilir.
  - Kalıp yapısı: `Context` (`ContextInterface()`), `strategy` üyesiyle soyut `Strategy`'ye (`AlgorithmInterface()`) bağlıdır; `ConcreteStrategyA/B/C` algoritmayı gerçekler.
  - Bileşenler: `Strategy` desteklenecek bütün algoritmaların ortak arayüzünü belirler; `ConcreteStrategyX` algoritma gerçeklemeleridir; `Context` algoritmaya ihtiyaç duyan nesnedir, bir ya da birkaç `ConcreteStrategy` ile ilişkilendirilir; `ContextInterface()` gerekli hallerde algoritma gerçeklemesine durum bilgisini açan metotlardır.
  - Gerçekleme ayrıntıları: Strategy ve Context seçilen algoritmayı gerçeklemek için etkileşebilir; Context'i kullanan istekçiler Context'e bir `ConcreteStrategy` nesnesi sağlar.
  - Kullanım: bir algoritmanın değişik çeşitlemelerine ihtiyaç duyulduğunda ve istekçilerin algoritmanın gerçeklenmesini bilmemesi gerektiğinde.
  - Zayıf yönler: istekçiler (ya da Context) farklı stratejilerin varlığından haberdar olmak zorundadır; Context'in açtığı durum bilgisinin tümüne her strateji ihtiyaç duymayabilir, bu da haberleşmede ek yük getirir.
- Strategy örneği: uluslararası kişisel finans programı (ekrandan, 00:37:10, 00:40:00; kaynak DP Explained)
  - Program değişik ülkelerin vergi kurallarıyla çalışmalıdır. Ana sınıf `TaskController`.
  - 1. çözüm (switch-case): `enum Country { TR, US, EU }`, `TaskController` içinde `code: Country`, `setCountry(Country)`, `taxAmount(double income)` içinde `switch (code)`. Hocanın anlattığı oranlar: TR %18, EU %15, US %12, diğerleri %10.
  - 2. çözüm (Strategy): `<<interface>> TaxRule` (`taxAmount(income: double): double`), bunu gerçekleyen `TRrules`, `EUrules`, `USrules`; `TaskController`'da `tax: TaxRule` ve `setTax(tax: TaxRule)`.
  - Vergi indirimi, aile desteği, kredi borcu hesabı gibi yeni işler eklenecekse iki seçenek vardır: aynı arayüze yeni metotlar eklemek (bir ülkenin bütün kuralları tek sınıfta toplanır) ya da her iş için Strategy kalıbını ayrıca kurmak (daha ince ayarlı bölme).
  - Strateji gerçeklemeleri iyi birer Singleton adayıdır (ekrandan, 00:45:10).
- State kalıbı (ekrandan, 01:04:50, 01:10:00)
  - Amaç: nesnenin iç durumu değiştiğinde davranışının da (büyük ölçekte, bütüncül olarak) değişmesini sağlamak.
  - Dörtlü Çete'nin örneği: TCP bağlantısı herhangi bir anda `Established`, `Listen` ya da `Closed` durumundadır ve her durumda farklı davranır. `TCPConnection` bir `TCPState` üyesi tutar; açma, kapama ve alındı verme (acknowledge) işlerini kendisi yapmaz, o

andaki duruma yaptırır.
  - Kalıp yapısı: `Context` (`Request()` içinde `state->Handle()`), soyut `State` (`Handle()`), `ConcreteStateA/B`. Request ve handle metotlarının adı aynı olmak zorunda değildir.
  - Durum geçişleri: Context kendi durumunu en iyi bildiği için geçişlerden genelde o sorumludur; ama somut durumlar da hangi koşulda hangi duruma geçileceğini bilip Context'i yeni duruma geçirebilir.
  - Kullanım: nesnenin çalışma şekli o andaki durumuna göre değişecekse; nesnenin durumuna bağlı karmaşık, uzun, çok kısımlı karar verme komutları gerekiyorsa.
  - Sonlu durum makinesi (FSM) benzetmesi: otomatanın her durumu kalıptaki bir somut durum sınıfına karşılık gelir; hangi girdide hangi duruma geçileceği ilgili somut durum sınıfına kodlanır.
- State ve Strategy farkı (ekrandan, 01:10:00)
  - State: Context'teki durum değiştikçe Context kendi `ConcreteState`'ini seçer; geçişler açıkça yapılır (Context'e verilen komutlarla ya da `ConcreteState`'lerin içinden); durum geçişleri daha sıktır.
  - Strategy: strateji geçişleri seyrektir; strateji nesneleri Context hakkında daha az şey bilir, kendilerine gelen parametrelere göre işlem yapar.
- State örneği: kişisel bilgi yöneticisi (PIM) (ekrandan, 01:17:50, 01:20:00)
  - Kayıtlar temiz (clean) ya da kirli (dirty) durumdadır. Kirli: değişiklik yapılmış ama kaydedilmemiş. Pencere kapatılırken kirliyse kaydetmek isteyip istemediğimiz sorulur. Eclipse'te değişmiş dosyanın adının başındaki yıldız da bunu gösterir.
  - Örnekte durum kayıt başına değil, bütün uygulama düzeyinde tutulur.
  - `PIMApplication`: `+CLEAN: State`, `+DIRTY: State` (public final static), `-currentState: State`, `getCurrentState()`, `setCurrentState(State)`.
  - Soyut `State` sınıfında ortak `save(File, Serializable)` ve varsayılan `edit()` kodlanmıştır. `CleanState.edit()` uygulamayı `DIRTY` durumuna geçirip `super.edit()` çağırır. `DirtyState` boş bir sınıftır (`extends State {}`).
  - Hoca, üst sınıftan türeyip hiçbir metot ya da alan eklemeyen sınıflara "marker (işaretçi) sınıf" dedi ve Java'daki `Serializable` arayüzünü örnek verdi.
- Template Method kalıbı (ekrandan, 01:30:00)
  - Amaç: bir algoritmanın ortak ve değişmemesi gereken adımlarını üst sınıfta belirleyip geri kalan adımların ayrıntısını alt sınıflara ertelemek. Strategy'de algoritma komple değişirken, alternatif algoritmaların ortak adımları varsa Template Method kullanılır.
  - Dörtlü Çete'nin örneği: farklı belge türleriyle çalışan uygulamalar için çerçeve (framework). `Document` (`Save()`, `Open()`, `Close()`, soyut `DoRead()`), `Application` (`AddDocument()`, `OpenDocument()`, soyut `DoCreateDocument()`, `CanOpenDocument()`, `AboutToOpenDocument()`), alt sınıflar `MyDocument` ve `MyApplication`.
  - `Application::OpenDocument` şablon metottur: önce `CanOpenDocument(name)` sorulur, açılamıyorsa döner; açılabiliyorsa `DoCreateDocument()` ile belge oluşturulur, `AddDocument` ile listeye eklenir, `AboutToOpenDocument(doc)` çağrılır, sonra `doc->Open()` ve `doc->DoRead()`.
  - Şablon metot alt sınıfta override edilmesin diye Java'da `final` yapılmalıdır.
  - Kalıp yapısı: soyut üst sınıfta şablon metot (sabit adımlar) ve değişebilen adımlar için soyut "primitive operation" metotları; somut alt sınıf bu işlemleri gerçekler.
  - Kullanım: algoritmanın değişmez kısımlarını bir kez kodlayıp değişen kısımları alt sınıflara bırakmak; alt sınıfların bazı davranışlarını denetlemek (alt sınıflara sınırlı esneklik tanımak); kardeş sınıfların ortak işlerini üst sınıfa çekmek.
- Template Method örneği: veritabanından bağımsız sorgu (ekrandan, 01:36:10; kaynak DP Explained)
  - Her VTYS için ortak beş adım: bağlantı komutunu biçimlendir, veritabanına bağlan, select komutunu biçimlendir, komutu gönder, sonuç kümesini al.
  - Strategy burada uygun değildir, çünkü Strategy'de işin hepsini strateji yapar; burada ortak adımlar vardır.
  - `QueryControl` (`-qt: QueryTemplate`, `QueryControl(QueryTemplate)`), soyut `QueryTemplate` (`+doQuery(DBname, querySpec: String) {final}`, `~formatConnect(DBname: String) {abstract}`, `~formatSelect(querySpec: String) {abstract}`), alt sınıflar `OracleQT`, `SQLServerQT`, `GraphDBQT`.
  - Komut biçimlendirme metotları iç çalışmayla ilgili olduğu için dışarıya açılmaz; slayta göre protected yapılabilir, kütüphane ya da çerçeve hazırlanmadığı için Java'da paket görünürlüğü de yeter.

- Visitor kalıbı (ekrandan, 02:00:00, 02:10:00, 02:10:20)
  - Amaç: bir nesne yapısını oluşturan elemanlar üzerinde yapılacak işlemleri tanımlamak.
  - Sorun: nesne yapısı her türden bağıntı içeren sınıfların örnekleri arasında kurulmuş olabilir; üzerinde işlem yapılan elemanların sınıf yapısını değiştirmek (ekstra metotlarla kalabalıklaştırmak) istemeyiz; zamanla yeni işler çıkabilir.
  - Eleştiri: madem yeni işlemler çıkıyor, eylemler ve sorumluluklar tam belirlenmemiş demektir; sınıf yapısını değiştirip farklı bir tasarım yapmak daha doğru olabilir. Bu her zaman mümkün değildir.
  - Örnek: derleyici. `x = y + 3` ifadesinin ayrıştırma ağacında atama, toplama ve operand düğümleri vardır. Düğümler üzerinde tip denetimi (type checking), kod üretimi ve düzgün yazım (pretty printing) yapılır.
  - Çözüm: `Node`'a `Accept(NodeVisitor)` eklenir. `NodeVisitor(VisitAssignment(AssignmentNode), VisitVariableRef(VariableRefNode))`, alt sınıfları `TypeCheckingVisitor`, `CodeGeneratingVisitor`. `AssignmentNode.Accept(v)` içinde `v->VisitAssignment(this)`, `VariableRefNode.Accept(v)` içinde `v->VisitVariableRef(this)`.
  - Kalıp yapısı: `Client`, soyut `Visitor` (`VisitConcreteElementA/B`), `ConcreteVisitor1/2`, `ObjectStructure`, soyut `Element` (`Accept(Visitor)`), `ConcreteElementA/B` (`Accept(Visitor v)`, `OperationA()/B()`).
  - Zayıf yönler: düğümler ziyaretçinin hangi metodunun kendi türü için olduğunu bilmek zorundadır (ek kullanma bağımlılığı); yeni bir somut eleman türü eklemek zordur, çünkü soyut `Visitor`'a ve bütün somut ziyaretçilere yeni bir `Visit...` metodu eklenmelidir.
  - Kullanım: değişik arayüzlere sahip nesnelerden oluşan bir yapıda nesneler üzerinde farklı işlemler gerekiyorsa ve yapıdaki türler değişmeyecekse ya da çok seyrek değişecekse.
- Visitor örneği: proje yönetim yazılımı (ekrandan, 02:20:00, 02:25:20, 02:30:00, 02:40:00; kaynak Applied Java Patterns)
  - Soyut `ProjectItem` (`name`, `description`), alt sınıfları `Deliverable` (`materialsCost`, `productionCost`), `Task` (`projectItems: ArrayList<ProjectItem>`, `timeRequired: double`), `Project` (`projectItems`). `Contact` görevin ve ürünün sahibidir. `DependentTask` başka görevlere bağımlı görevdir.
  - 1. çözüm: `ProjectCalculator` (`calculateTimeRequired()`, `calculateCost()` ve alt görevler için özel metotlar) ve `Report`.
  - 2. çözüm: her sınıfa `accept(ProjectVisitor)` eklenir. `<<interface>> ProjectVisitor` (`visitDependentTask`, `visitDeliverable`, `visitTask`, `visitProject`), gerçeklemeleri `ProjectCostVisitor` ve `ProjectTimeVisitor`.
  - Örnek veride proje bir yüksek lisans eğitimidir: dönemler görev (task), alınan dersler, tez metni ve bildiri ürün (deliverable). İki çözüm de aynı sonucu verdi: 60 hafta ve 1174 saat çalışma.

Hocanın Özellikle Vurguladığı Kısımlar

- Sınıf şemaları güncel standarda göre çizilmeli
  - Slaytlardaki Dörtlü Çete şemaları eski gösterimle çizilmiştir. Hoca şemaların güncel UML 2.0 standardına göre çizilmesi gerektiğini hatırlattı.
- Üst sınıf alt sınıflarını bilmemeli
  - Strateji soyut sınıfına en uygun alt sınıfı öneren bir karar metodu koymak tavsiye edilmez; kalıtımda bağımlılık alt sınıftan üst sınıfa doğrudur, tersi kalıtımın özüne aykırıdır.
- Switch-case ve instanceof birer uyarı işaretidir
  - Kendinizi switch-case kullanırken görüyorsanız kalıtım ve çok biçimlilik kullanmanız gerekir; gerçek uygulamada her case yüzlerce satır olur ve bakım pahalılaşır.
  - `instanceof` kullanıyorsanız büyük ihtimalle kalıtım hiyerarşisini doğru kuramamışsınızdır. Visitor çözümünün somut avantajı budur.
- Sınıf şemaları aynı olabilir, fark ayrıntıdadır
  - Builder ile Strategy'nin sınıf şemaları neredeyse aynıdır. Builder'da Director işi adım adım alt işlere bölüp yaptırır; Strategy'de iş komple stratejiye verilir.
  - State ile Strategy'nin en önemli farkı geçiş sıklığıdır: strateji uzun vadelidir ve seyrek değişir, durum sık değişir.

- Ortak adım varsa Template Method
  - Farklı veritabanları için akla önce Strategy gelebilir; ama "yürütülecek ortak işlemler de var" ise anahtar kelime budur ve Template Method kurulur.
- Şablon metot `final` olmalı
  - Özenle sıralanmış adımları biri acemilikten ya da kötü niyetle override edip bozmasın diye.
- Visitor ne zaman seçilir
  - Yeni eleman türü eklenmeyecek ama yeni işler eklenecekse uygundur; eleman türleri sık değişecekse uygun değildir.
- Sequence diyagramında dönüş okları
  - Her düz çizgili çağrı okunun karşılığında kesikli dönüş oku çizilmelidir; yoksa diyagram iyi anlaşılmaz.

Kısa Tekrar Notları

- Strategy: aynı işin farklı algoritmaları; `Context → Strategy.AlgorithmInterface() → ConcreteStrategy`. Çalışma anında değiştirilebilir.
- Hangi stratejinin kullanılacağına Context de istekçi (client) de karar verebilir; ayrı bir fabrika ya da yapılandırma dosyası da kullanılabilir.
- Strategy'nin zayıf yönleri: istekçi stratejileri bilmeli; Context bütün durumunu açar, uzak makinelerde haberleşme yükü olur.
- Vergi örneği: switch-case yerine `TaxRule` arayüzü ve ülke başına bir sınıf; yeni ülke = yeni sınıf. Strateji sınıfları Singleton adayıdır.
- State: davranış durumla bütüncül olarak değişir; `Context.Request() → state->Handle()`.
- State ve Strategy: durum geçişleri sık, strateji geçişleri seyrek; durum Context'i daha sık sorgular; Context kendi durumunu seçer.
- PIM örneği: temiz ve kirli durumlar; edit kirliye, save temize geçirir; `DirtyState` bir marker sınıftır.
- Template Method: şablon metot sabit adımları kodlar, değişen adımlar soyut metotlarla alt sınıfa bırakılır; şablon metot `final`.
- Sorgu örneği: `QueryTemplate.doQuery` final, `formatConnect` ve `formatSelect` soyut ve paket görünürlüğünde.
- Visitor: `Element.Accept(Visitor v) → v.VisitConcreteElementX(this)`. Yeni işlem = yeni ziyaretçi sınıfı; yeni eleman türü = bütün ziyaretçilere yeni metot.
- Proje örneği: `ProjectCostVisitor` ve `ProjectTimeVisitor`; `instanceof` denetimi ortadan kalktı.

Detaylı Açıklamalar

Strategy

Kalıp yapısı (ekrandan, 00:24:30):

```
Context <>------- strategy ------> Strategy
  ContextInterface()                 AlgorithmInterface()
                                           ^
                     +---------------------+---------------------+
                     |                     |                     |
            ConcreteStrategyA     ConcreteStrategyB     ConcreteStrategyC
            AlgorithmInterface()  AlgorithmInterface()  AlgorithmInterface()
```

Hoca, şemadaki parça-bütün ilişkisinin, parçanın (stratejinin) ait olduğu bütünü (Context'i) bildiği bir iletişim olarak okunmasını istedi: somut strateji gerekirse Context'e başvurup ek durum bilgisine ulaşabilir. Stratejinin Context'e en uygun gerçeklemeyi önermesi de mümkündür, ama bu soyut strateji sınıfının alt sınıflarını bilmesini gerektirdiği için önerilmez. Buna karşılık Context'in olası stratejileri bilmesinde ve en uygununu kendisinin seçmesinde sakınca yoktur.

Context ile strateji farklı adres uzaylarındaysa (örneğin farklı makinelerde) ve haberleşme gecikmesi yüksekse, Context'in bütün durum bilgisini açması, bu bilginin tamamına ihtiyaç

duymayan stratejiler için ek haberleşme yükü demektir. Hoca bunun her strateji için geçerli olmadığından çok ciddi bir zayıflık sayılmayacağını söyledi.

Vergi örneğinin 1. çözümü (ekrandan, 00:37:10):

```
package strategy.solution1;
public enum Country { TR, US, EU }
public class TaskController {
    private Country code;
    public void setCountry(Country code){
        this.code = code;
    }
    public double taxAmount(double income){
        switch (code) {
            case TR: return income * 0.18;
            case EU: return income * 0.15;
            case US: return income * 0.12;
            default: return income * 0.10;
        } } }
```

Bu çözümde yeni bir ülke (örneğin Birleşik Krallık) eklemek için hem enum'a yeni değer hem switch'e yeni case eklemek gerekir ve `taxAmount` her seferinde daha uzun, daha karmaşık hale gelir. 2. çözümde ise yalnız `TaxRule`'u gerçekleyen yeni bir sınıf yazılır; Türkiye'nin kuralı değişirse yalnız `TRrules` düzenlenir.

Singleton olarak yazılmış strateji (ekrandan, 00:45:10):

```
package singleton.example;
public class TRrules implements TaxRule {
    private static TRrules instance;
    private TRrules( ) {
        //gerekli ilklendirmeler
    }
    public static TRrules getInstance() {
        if( instance == null )
            instance = new TRrules( );
        return instance;
    }
    public double taxAmount(double income) {
        return income * 0.18;
    }
}
```

`EUrules` ve `USrules` de benzer biçimde gerçeklenir. Hoca ek karmaşıklığın değip değmeyeceğinin duruma göre değerlendirileceğini söyledi: strateji nesneleri setter'larla farklı parametrelere ayarlanabiliyorsa ve farklı şirketlerin vergisi farklı ayarlı örneklerle hesaplanıyorsa, "Türkiye kuralı tektir" diyerek eş biçimliliği Singleton ile sağlamak anlamlıdır.

State

TCP örneğinde bağlantı kapalıyken artık dinleme yapamaz, yalnız yeniden kurulabilir; dinleme durumunda farklı sinyallere farklı yanıt verir. Hoca, bu bütüncül değişikliğin switch-case ile de kodlanabileceğini ama bunun esnek olmadığını önceki kalıplarda gördüğümüzü söyledi. İlke yine "değişeni bul ve sarmala"dır: değişen şey soketin durumudur.

Kalıp yapısı (ekrandan, 01:04:50):

```
Context <>------- state -------> State
  Request()                        Handle()
  // state->Handle()                 ^
                                     |
                              +------+------+
                              |             |
                      ConcreteStateA   ConcreteStateB
                        Handle()         Handle()
```

Hoca State ile Strategy arasındaki farkı üç maddede topladı: (1) en önemlisi, strateji geçişleri seyrek, durum geçişleri sıktır; "strateji" sözcüğünün kendisi de uzun vadeli bir oyun planını anlatır; (2) strateji nesneleri daha çok kendilerine verilen parametrelerle çalışır, durum nesneleri ise Context'i daha sık sorgular, çünkü durum varlığın üye alanlarının değerlerine çok bağlıdır; (3) State'te Context genelde kendi somut durumunu seçer.

PIM örneğinde `CleanState` (ekrandan, 01:17:50):

```
package dp.state.example;
import java.io.*;
public class CleanState extends State {
    public void edit( ){
        PIMApplication.setCurrentState(PIMApplication. ... );
        super.edit();
    }
    public void save(File f, Serializable s) throws IO... {
        super.save(f, s);
        PIMApplication.setCurrentState(PIMApplication. ... );
    }
}
```

```
package dp.state.example;
public class DirtyState extends State { }
```

Hoca, `State` sınıfındaki boş `edit` metodunun kod kalitesi aracının uyarısına göre soyut olması gerektiğini kabul etti; ama iki durumda da düzenleme temelde aynı iş olduğu için varsayılan davranışı üst sınıfa koymayı tercih ettiğini, gerçek uygulamada bu metot doldurulunca uyarının kalkacağını söyledi.

> **Ders sonrası not:** Hoca serileştirmeyi nesnenin durum bilgisinin sınıfın kaynak ya da bayt koduyla birlikte dosyaya yazılması diye anlattı. Java serileştirmesi büyük olasılıkla nesnenin alan değerlerini ve sınıfı tanıtan bir betimleyiciyi (sınıf adı, `serialVersionUID`) yazar, sınıfın bayt kodunu yazmaz.

Template Method

Dörtlü Çete'nin şablon metodu (ekrandan, 01:30:00):

```
void Application::OpenDocument (const char* name) {
    if (!CanOpenDocument(name)) {
        // cannot handle this document
        return;
    }
    Document* doc = DoCreateDocument();
    if (doc) {
        _docs->AddDocument(doc);
        AboutToOpenDocument(doc);
        doc->Open();
        doc->DoRead();
    }
}
```

`OpenDocument` somut bir metottur ama içinde soyut metotlar çağrılır. `MyApplication` bu soyut metotları (`DoCreateDocument` içinde `return new MyDocument`) gerçekler; `MyDocument` de `DoRead`'i gerçekler. Böylece çerçevenin kullanıcısı yalnız kendi belge türünün nasıl okunacağını ve oluşturulacağını kodlar.

Sorgu örneğinin sınıf şeması (ekrandan, 01:36:10):

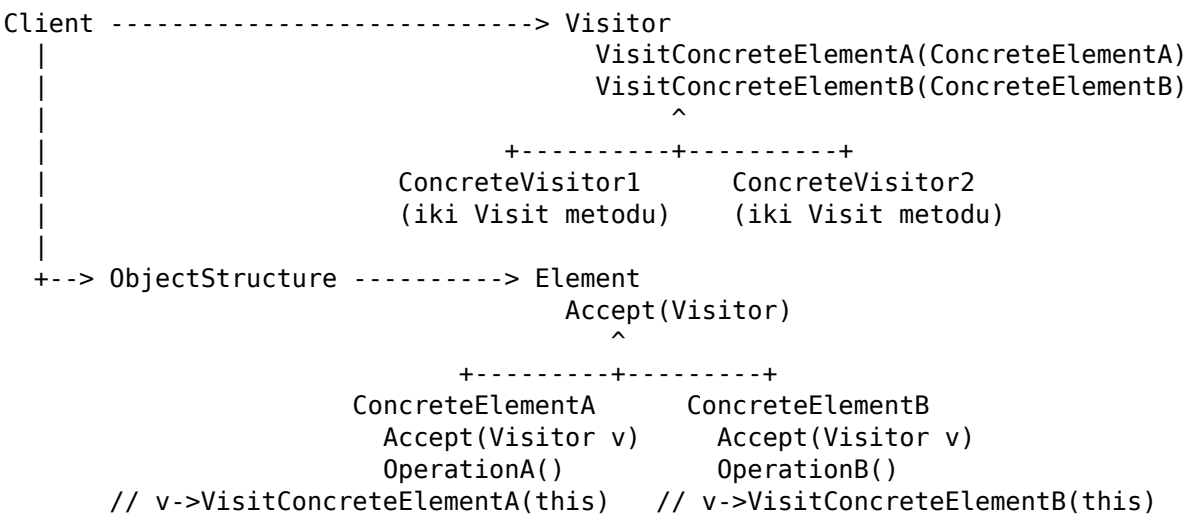
```
QueryControl                  1      1  <<abstract>> QueryTemplate
- qt : QueryTemplate     ------------>  + doQuery(DBname, querySpec : String) {final}
+ QueryControl(QueryTemplate)          ~ formatConnect(DBname : String) {abstract}
                                       ~ formatSelect(querySpec : String) {abstract}
                                                  ^
```



Hoca soyut biçimlendirme metotlarını paket görünürlüğünde tanımladığını söyledi: veritabanına özgü bu parçaları tek başına çağırmanın anlamı yoktur, ama aynı paketteki yardımcı sınıflar gerekirse çağırabilsin diye `private` de yapılmamıştır.

Visitor

Kalıp yapısı (ekrandan, 02:10:20):



Hoca derleyici örneğinde önceki tasarımı şöyle eleştirdi: tip denetimi, kod üretimi ve düzgün yazım her düğüm sınıfına dağıtılırsa, örneğin tip denetimindeki bir düzeltme için bütün düğüm sınıfları tek tek dolaşılmalıdır; bu bir bakım kâbusudur. Visitor ile tip denetimine ilişkin bütün kod, hangi düğüm türüyle ilgili olursa olsun `TypeCheckingVisitor` içinde toplanır. Her somut düğüm sınıfından `NodeVisitor`'a, şemada çizilmemiş bir kullanma bağımlılığı (kesikli ok) vardır.

Etkileşim sırası: nesne yapısı bir elemana ziyaretçiyi verip `Accept` çağırır; eleman, ziyaretçinin kendi türüne ait `Visit` metodunu kendisini parametre vererek çağırır; ziyaretçi o eleman üzerindeki işlemleri (örneğin `OperationA`) sırayla yaptırır; işlemler bitince `Visit`, sonra `Accept` döner ve sıradaki elemana geçilir.

Proje örneğinde 1. çözümün `ProjectCalculator`'ı ile 2. çözümün `ProjectCostVisitor`'ı (ekrandan, 02:40:00):

```
package dp.visitor.solution1;
public class ProjectCalculator {
    private Project project;
    public ProjectCalculator(Project project) {
        this.project = project;
    }
    public double calculateTimeRequired( ) {
        double time = 0.0;
        for( ProjectItem item : project.getProje... ) {
            if( item instanceof Task ) {
                time += calculateTimeRequired((T...
            }
        }
        ...
```

```
package dp.visitor.solution2;
public class ProjectCostVisitor implements Pro... {
    private double cost = 0.0;
    public ProjectCostVisitor(Project project) {
```

```
        project.accept(this);
    }
    public void visitDependentTask(DependentTa... p) {
        visitTask(p);
    }
    ...
```

Hoca satır sayılarını da karşılaştırdı: `ProjectCalculator` yaklaşık 50 satır, iki ziyaretçinin her biri yaklaşık 35 satırdır. Bir hata çıktığında sorunun zamanda mı maliyette mi olduğu bilinip yalnız ilgili ziyaretçi incelenir. Asıl fark ise yeni bir hesaplama gerektiğinde yalnız yeni bir ziyaretçi sınıfı eklenmesi ve `instanceof` denetiminin ortadan kalkmasıdır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.